

Ridge and Lasso Regression on AutoMPG

ScalaTion and `statsmodels`

CSCI 4360 Data Science ML II

October 6, 2026

Abstract

We fit Ridge and Lasso to the AutoMPG data in ScalaTion and in `statsmodels`. Each package also gets a plain least squares (OLS) fit to compare against. Both packages use the same train/test split, the same cross-validation folds, the same scaling and the same list of penalty values. Ridge gives the same coefficients in both packages (they match to about 10^{-8}). Lasso differs by at most 0.07 because ScalaTion only solves it approximately. All six models have a test RMSE between 3.23 and 3.28 mpg, and the two Lasso models are the best of the six. Weight and model year explain almost all of the fuel economy.

1 Data

We used the UCI Auto MPG data [1]. The response is fuel economy in miles per gallon (`mpg`). The six predictors are `cylinders`, `displacement`, `horsepower`, `weight`, `acceleration` and `model_year`. Six cars have no `horsepower`, so we dropped them and kept 392 rows. We left out the `origin` column, as ScalaTion's own `Example_AutoMPG` does, because it is a category code that can overlap with the other predictors. The ScalaTion run uses ScalaTion's built-in copy of the data. The program checks that its totals match the UCI file ($\sum \text{mpg} = 9190.8$, $\sum \text{weight} = 1\,167\,213$), and the ScalaTion OLS fit matches the `statsmodels` OLS fit to 10^{-8} , so the rows and their order are the same.

Variable	Mean	Std. dev.	Min	Max	VIF
mpg (response)	23.45	7.81	9.0	46.6	n/a
cylinders	5.47	1.71	3.0	8.0	10.99
displacement	194.41	104.64	68.0	455.0	19.30
horsepower	104.47	38.49	46.0	230.0	9.39
weight	2977.58	849.40	1613.0	5140.0	10.06
acceleration	15.54	2.76	8.0	24.8	2.68
model_year	75.98	3.68	70.0	82.0	1.27

Table 1: The 392 cars. VIF (variance inflation factor) is computed on the 314 training rows.

Table 1 shows that the engine-size variables move together. The VIFs of `displacement`, `cylinders`, `weight` and `horsepower` are all about 9 or higher, and `cylinders` and `displacement` have a correlation of 0.95. In this situation the OLS coefficients are unstable even though the model predicts well. Ridge and Lasso are meant for this.

2 Method

Split and folds. We split the 392 rows into 314 training rows and 78 test rows (seed 42). We picked the penalty using only the training rows, with 5-fold cross-validation. The Python script saves the test rows and the fold of every training row to `data/test_indices.csv` and `data/cv_folds.csv`. The ScalaTion program reads the same files, so both packages see the same data at every step.

Scaling. Each predictor is scaled to mean 0 and standard deviation 1, using the training rows only (inside each CV fold, using only that fold’s training rows). The intercept is never penalized. Coefficients are therefore in “mpg per one standard deviation of the predictor”. Table 6 gives them in the original units too.

Penalties. With n training rows, scaled predictors $\mathbf{z}_i$ and intercept β_0 , ScalaTion minimizes

$$\begin{aligned} \text{Ridge: } & \sum_i (y_i - \beta_0 - \mathbf{z}_i^\top \boldsymbol{\beta})^2 + \lambda \|\boldsymbol{\beta}\|_2^2, \\ \text{Lasso: } & \frac{1}{2} \sum_i (y_i - \beta_0 - \mathbf{z}_i^\top \boldsymbol{\beta})^2 + \lambda \|\boldsymbol{\beta}\|_1. \end{aligned}$$

`statsmodels` minimizes $\frac{1}{2n} \text{RSS} + \alpha [\frac{1-w}{2} \|\boldsymbol{\beta}\|_2^2 + w \|\boldsymbol{\beta}\|_1]$, with $w = 0$ for Ridge and $w = 1$ for Lasso. Multiply by n and the two forms are the same when

$$\lambda = n \alpha \quad (\text{Ridge and Lasso}).$$

We report every penalty as $\lambda = n_{\text{train}} \alpha$ with $n_{\text{train}} = 314$.

Tuning. Both packages try the same 60 values of α , spaced evenly on a log scale from 10^{-4} to 10 (so λ runs from 0.031 to 3140). We keep the value with the lowest 5-fold CV RMSE. The same list of values gives the coefficient paths.

statsmodels. We used `sm.OLS(...).fit_regularized(method="elastic_net")` (version 0.15.0). Two things needed care. First, it shrinks every column, including the intercept, unless you give a separate penalty for each coefficient. With one penalty value, the Ridge intercept dropped from 23.4 to 15.6 in a quick test. We give the intercept a penalty of 0. Second, the Lasso solver never looks at a coefficient again once it reaches zero (the source code says this), so one call can stop too early. We run it again from its last answer until nothing changes. The unit tests check the result against scikit-learn’s Lasso and against the optimality conditions (Section 4).

ScalaTion. We used `RidgeRegression.rescale` and `LassoRegression.rescale`. They scale X with `NormForm` and center y , so the intercept is the training mean of `mpg` and is not penalized. Ridge is solved directly (Cholesky). Lasso is solved with ADMM (`LassoAdmm`). The two classes share one hyper-parameter object, so the program sets λ right before it builds each model and clears the ADMM warm-start cache before each Lasso fit. The OLS fit is ScalaTion’s `Regression` on $[\mathbf{1}, \mathbf{Z}]$. We wrote the cross-validation loop ourselves (fit on four folds, predict the fifth) instead of using ScalaTion’s `findLambda`, because `findLambda` only doubles λ from a start value and cannot use our list of values or our folds.

Metrics. RMSE, R^2 and MAE on the 314 training rows and on the 78 test rows, plus the 5-fold CV RMSE (the square root of the mean fold MSE).

3 Results

3.1 Overview

Table 2 lists all six models. Figure 1 shows the CV curves, Table 3 the coefficients, and Figures 2 and 3 the coefficient paths.

Software	Model	λ	Zero coef.	CV RMSE	Train R^2	Test R^2	Test RMSE	Test MAE
statsmodels	OLS	n/a	0/6	3.5852	0.8105	0.7945	3.2526	2.5068
statsmodels	Ridge	5.02	0/6	3.5747	0.8097	0.7914	3.2769	2.5190
statsmodels	Lasso	19.66	1/6	3.5414	0.8104	0.7960	3.2404	2.4870
ScalaTion	OLS	n/a	0/6	3.5852	0.8105	0.7945	3.2526	2.5068
ScalaTion	Ridge	3.39	0/6	3.5747	0.8101	0.7923	3.2700	2.5137
ScalaTion	Lasso	29.04	0/6	3.5387	0.8103	0.7969	3.2332	2.4773

Table 2: Fit and prediction results. $\lambda = n_{\text{train}}\alpha$ is the penalty picked by 5-fold CV. “Zero coef.” counts slopes that are exactly zero (out of 6). Test columns use the 78 test cars. CV RMSE uses the 314 training cars.

Term	statsmodels			ScalaTion		
	OLS	Ridge	Lasso	OLS	Ridge	Lasso
intercept	23.6102	23.6102	23.6102	23.6102	23.6102	23.6102
cylinders	−0.1931	−0.2752	−0.1509	−0.1931	−0.2434	−0.1485
displacement	0.0810	−0.3515	0.0000	0.0810	−0.2421	−0.0030
horsepower	−0.0874	−0.4120	−0.1671	−0.0874	−0.3243	−0.2048
weight	−5.5169	−4.7868	−5.3947	−5.5169	−4.9876	−5.3513
acceleration	0.1663	−0.0098	0.0902	0.1663	0.0360	0.0597
model_year	2.7513	2.6577	2.7000	2.7513	2.6849	2.6764

Table 3: Coefficients on the scaled predictors (mpg per standard deviation of the predictor). The intercept is the training mean of `mpg`. OLS is the same in both packages, so the data, split and scaling agree.

3.2 Ridge, statsmodels

CV picks $\lambda = 5.02$ ($\alpha = 0.0160$). Ridge keeps all six predictors. The `weight` coefficient gets smaller, from -5.52 in OLS to -4.79 . The three engine-size variables are small and mixed in sign in OLS ($-0.19, +0.08, -0.09$). Ridge turns them into a steady negative group ($-0.28, -0.35, -0.41$) for `cylinders`, `displacement` and `horsepower`. So Ridge spreads the effect of a bigger engine over the related variables, instead of letting one coefficient take all of it. `acceleration` shrinks to about zero (-0.01) and `model_year` barely changes ($2.75 \rightarrow 2.66$). The predictions are about the same as OLS: CV RMSE 3.5747 against 3.5852, test RMSE 3.277 against 3.253, test R^2 0.791 against 0.794. With 314 rows and six predictors, Ridge does not cut the error much. What it gives us is more believable coefficients, not better predictions.

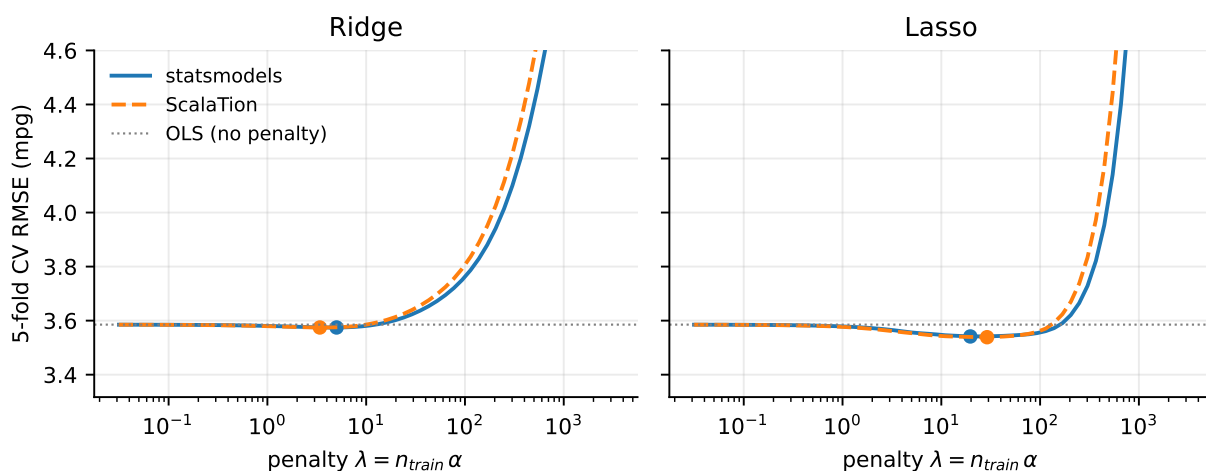


Figure 1: 5-fold CV RMSE against the penalty $\lambda = n_{\text{train}} \alpha$. Dots mark the chosen λ . The Ridge curve is almost flat up to $\lambda \approx 10$. The Lasso curve has a shallow low point near $\lambda \approx 20$ to 30 .

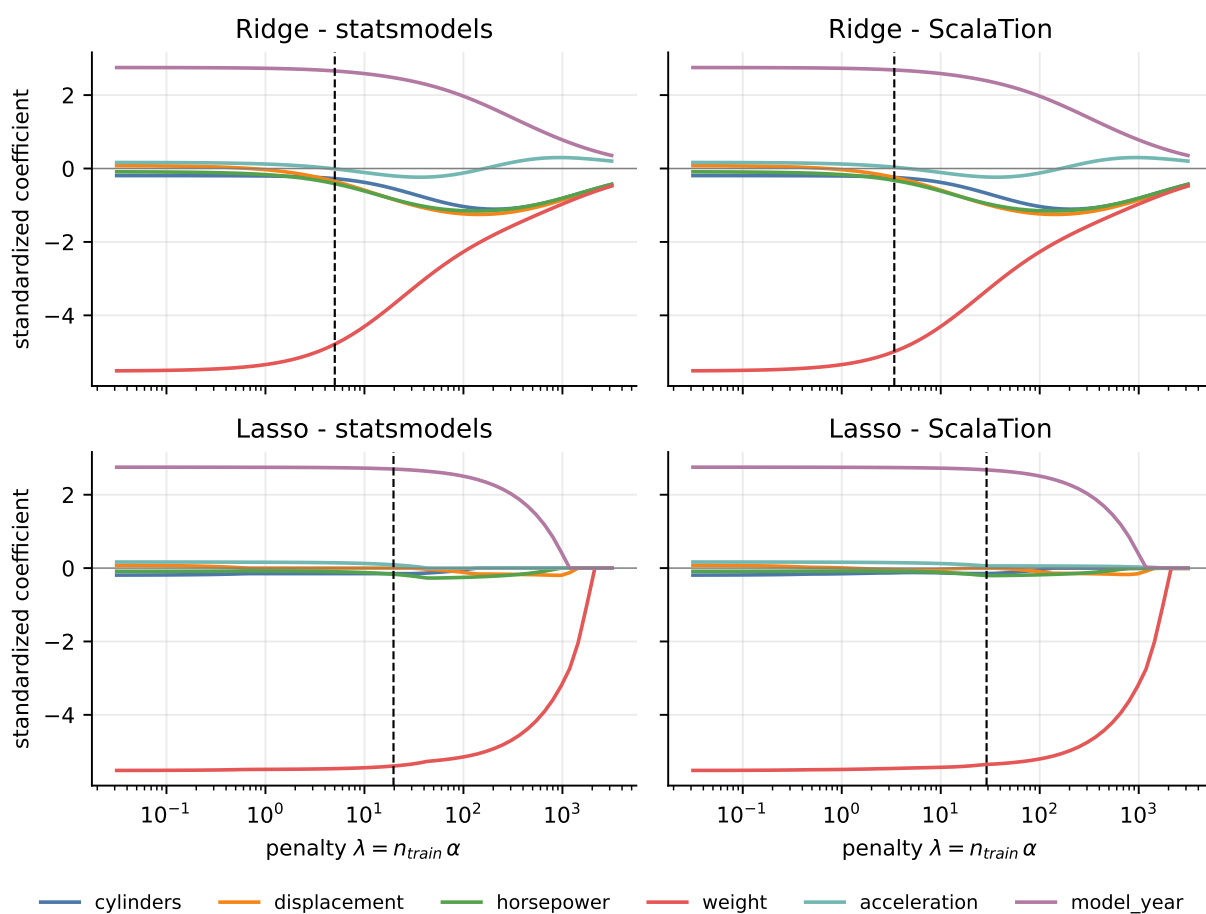


Figure 2: Coefficient paths. Dashed lines mark the chosen λ . Ridge shrinks every coefficient smoothly and none reach zero. Lasso sets coefficients to zero (**statsmodels**) or close to zero (**ScalaTion**).

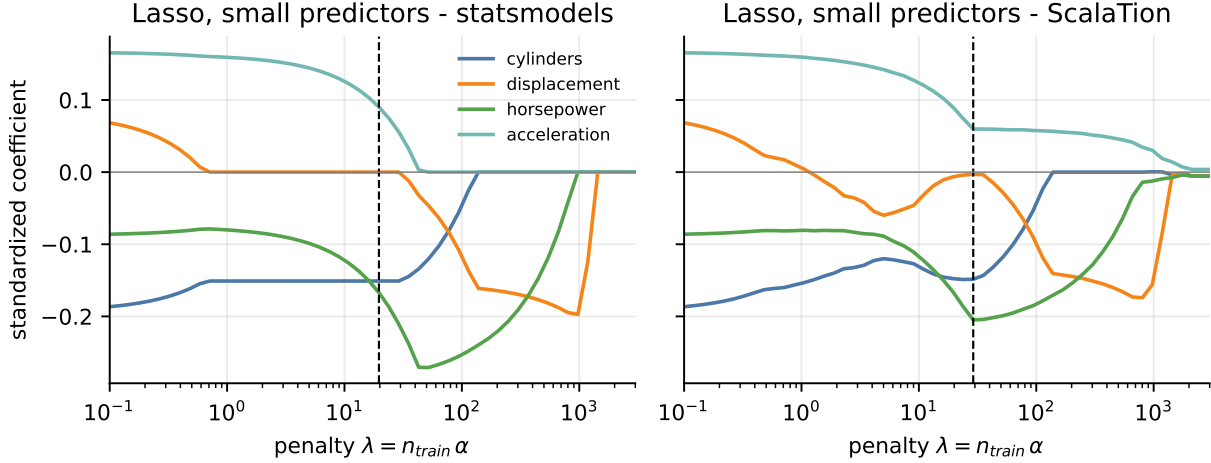


Figure 3: Lasso paths for the four small predictors (`weight` and `model_year` left out so the scale is readable).

3.3 Lasso, statsmodels

CV picks $\lambda = 19.66$ ($\alpha = 0.0626$). `displacement` is set exactly to zero. The other five stay but shrink: `weight` = -5.39 (OLS -5.52), `model_year` = 2.70 , `horsepower` = -0.17 , `cylinders` = -0.15 , `acceleration` = 0.09 . Displacement is probably dropped because it overlaps the most with the others (VIF 19.3, correlation 0.95 with `cylinders`), so `cylinders`, `horsepower` and `weight` already carry what it knows. This is the best `statsmodels` model: CV RMSE 3.5414 (OLS 3.5852), test RMSE 3.240 (OLS 3.253), test R^2 0.796. Lasso beat OLS in all five CV folds (by 0.03 to 0.11 mpg). Ridge beat OLS in only three. The gain is steady but small next to the noise in a 78-row test set. We also refit OLS on the five kept predictors (Table 4). Only `weight` ($t = -9.7$) and `model_year` ($t = 12.4$) are clearly different from zero. `cylinders`, `horsepower` and `acceleration` have $p > 0.6$. These p -values ignore that the variables were picked, so they look better than they should. They still fit the paths in Figures 2 and 3. As λ grows past the chosen value, the `statsmodels` Lasso drops `acceleration` (at $\lambda \approx 50$), `cylinders` (≈ 130), `horsepower` (≈ 900), `model_year` (≈ 1100) and `displacement` (≈ 1300). `weight` is the last one left (≈ 1900).

Term	Coef.	Std. err.	t	p -value	95% CI low	95% CI high
intercept	23.6102	0.1968	119.99	0.0000	23.2230	23.9974
cylinders	-0.1510	0.4730	-0.32	0.7498	-1.0817	0.7798
horsepower	-0.0754	0.5900	-0.13	0.8984	-1.2363	1.0854
weight	-5.4915	0.5634	-9.75	0.0000	-6.6001	-4.3829
acceleration	0.1628	0.3202	0.51	0.6114	-0.4673	0.7930
model_year	2.7498	0.2219	12.39	0.0000	2.3132	3.1865

Table 4: `statsmodels`: plain OLS refit on the predictors Lasso kept (scaled predictors). The p -values ignore the selection step.

3.4 Ridge, ScalaTion

CV picks $\lambda = 3.39$ ($\alpha = 0.0108$), with the same CV RMSE as `statsmodels` (3.5747). The coefficients ($-0.24, -0.24, -0.32, -4.99, +0.04, +2.68$) follow the same pattern as in `statsmodels`:

weight shrinks a little, the three engine-size variables become a steady negative group, and `acceleration` is close to zero. Test RMSE is 3.270 and test R^2 is 0.792, within 0.02 mpg of OLS (3.253). These coefficients differ from the `statsmodels` Ridge column only because the two searches landed on different points of an almost flat CV curve ($\lambda = 3.39$ against 5.02, both with CV RMSE 3.5747). At the *same* λ the two packages agree to 8×10^{-9} along the whole path (Table 5).

3.5 Lasso, ScalaTion

CV picks $\lambda = 29.04$ ($\alpha = 0.0925$). `displacement` shrinks to -0.003 . That is zero for practical purposes (under 0.01), but ScalaTion does not call it an exact zero (Table 2). Its ADMM solver stops at a loose tolerance, and its clean-up step only zeroes coefficients below 10^{-3} of the largest one (0.0027 here). The other coefficients (`cylinders` = -0.15 , `horsepower` = -0.20 , `weight` = -5.35 , `acceleration` = 0.06 , `model_year` = 2.68) tell the same story as `statsmodels`: weight and model year matter most and the engine-size variables are small. This is the best of the six models by test error (RMSE 3.233, R^2 0.797) and by CV RMSE (3.5387). ScalaTion’s own coefficient table for Ridge and Lasso prints standard errors, t values and p values that assume no penalty, so they are not valid here and we do not use them (the raw output is in Appendix A).

3.6 The two packages compared

Quantity	OLS	Ridge	Lasso
Max coefficient gap over the whole λ grid	n/a	7.6e-09	7.2e-02
Median coefficient gap over the grid	n/a	6.6e-09	2.0e-02
Max coefficient gap, tuned final models	7.6e-09	2.0e-01	4.3e-02
Test-RMSE gap, tuned final models	4.4e-16	6.9e-03	7.2e-03

Table 5: ScalaTion minus `statsmodels`, in scaled coefficient units. The path rows compare the two packages at the *same* λ on all 60 grid points. The final-model rows also include the effect of the two searches picking different λ .

- **OLS and Ridge agree to about 10^{-8} .** The OLS coefficients differ by 8×10^{-9} and the Ridge paths by 8×10^{-9} . This also confirms the $\lambda = n\alpha$ link and the matching scaling.
- **Lasso differs by at most about 0.07 (median 0.02).** Our `statsmodels` code (with restarts) gives the exact Lasso answer, as the unit tests show. ScalaTion’s `LassoAdmm` has fixed tolerances (10^{-4} absolute, 10^{-2} relative) that user code cannot change, so its answer is approximate. The ScalaTion path in Figure 3 is less smooth than the exact one. The gap is about 1% of the largest coefficient and changes no conclusion.
- **All six models predict about equally well.** Test RMSE runs from 3.233 to 3.277, a spread of 1.3%. With 78 test rows, the standard error of an RMSE is roughly 0.26 mpg, so none of these differences is meaningful. Do not read much into the ranking.

3.7 What the fits say about fuel economy

In every model, heavier cars get lower mpg (about -4.8 to -5.5 mpg per standard deviation of weight, which is roughly 850 lb), and newer model years get higher mpg (about $+2.7$ mpg per

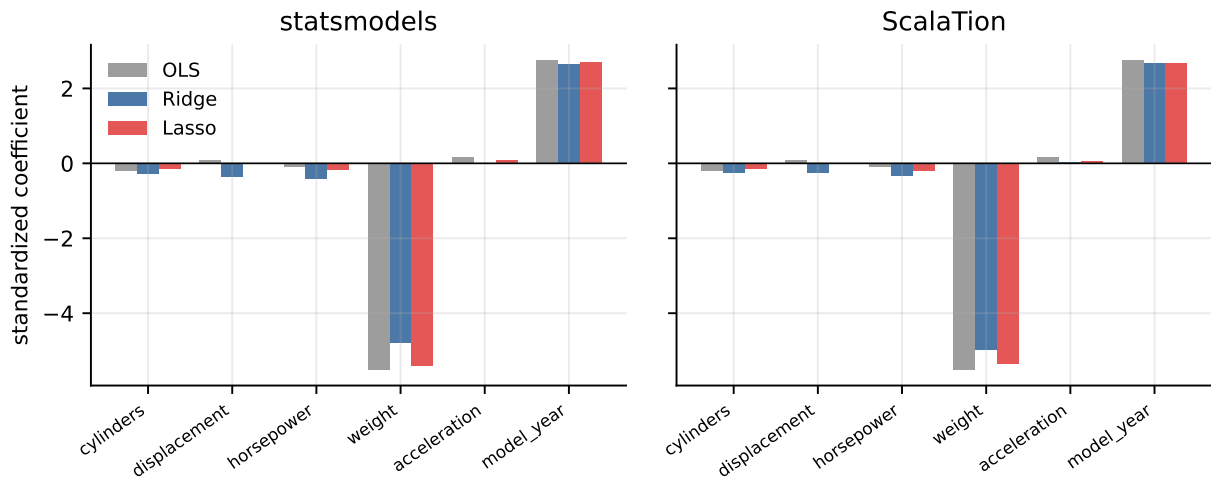


Figure 4: Coefficients of the OLS, Ridge and Lasso fits in each package (scaled predictors).

Term	statsmodels			ScalaTion		
	OLS	Ridge	Lasso	OLS	Ridge	Lasso
intercept	-14.59995	-12.29165	-13.30611	-14.59995	-12.95043	-12.69685
cylinders	-0.11361	-0.16191	-0.08880	-0.11361	-0.14320	-0.08734
displacement	0.00078	-0.00339	0.00000	0.00078	-0.00233	-0.00003
horsepower	-0.00229	-0.01077	-0.00437	-0.00229	-0.00848	-0.00536
weight	-0.00656	-0.00569	-0.00641	-0.00656	-0.00593	-0.00636
acceleration	0.05911	-0.00349	0.03206	0.05911	0.01280	0.02122
model_year	0.75560	0.72988	0.74152	0.75560	0.73736	0.73503

Table 6: The same coefficients in original units, which is mpg per unit of each predictor. For example, the statsmodels Lasso gives -0.0064 mpg per pound (about -6.4 mpg per 1000 lb) and $+0.74$ mpg per model year.

standard deviation, roughly 3.7 years). Once these two are in the model, cylinders, displacement, horsepower and acceleration add little. They move together with weight, so their own coefficients are small and change with the penalty. A model with just `weight` and `model_year` already explains most of the variance. The penalty changes how the credit is split among related predictors much more than it changes the predictions.

4 Checks

The Python code has 11 automatic tests (`python -m unittest discover -s tests`) and all pass. They check that: the data has 392 clean rows and the right mpg total; the split has no overlap, covers every row and gives the same result each time; scaled columns have mean 0 and standard deviation 1; Ridge equals the closed-form answer $(Z^T Z + n\alpha D)^{-1} Z^T y$ and scikit-learn's Ridge with penalty $n\alpha$; Lasso equals scikit-learn's Lasso at 10 values spread over the grid (within 10^{-6}) and meets the optimality (KKT) conditions; the intercept is not penalized; a tiny penalty gives back OLS and a huge Lasso penalty zeroes every slope; and coefficients converted to original units give the same predictions. The ScalaTion program stops with an error if its data or training rows do not match the UCI data, and its OLS fit matches the `statsmodels` OLS fit. ScalaTion's variance inflation factors for the six predictors (10.99, 19.30, 9.39, 10.06, 2.68, 1.27) match the ones we computed separately in Python (Table 1).

5 Limits

- We used one train/test split and one set of CV folds. The CV curves are very flat, so the best λ is not well pinned down. That is why the two packages picked different λ values with the same CV RMSE.
- ScalaTion's built-in `crossValidate` reuses one scaling for every fold, which lets a little information from the validation rows leak in. Our own CV loop redoes the scaling inside each fold, in both packages.
- ScalaTion's Lasso is approximate, so it gives a near-zero value instead of an exact zero for `displacement`.
- The p -values in Table 4 do not account for picking the variables first.

6 How to run

```
cd Project2/python
pip install -r requirements.txt
python regularized_regression.py      # statsmodels run (also writes the split/fold CSVs)
python -m unittest discover -s tests # 11 tests
# ScalaTion (needs JDK 21 and sbt; Scala 3.8.3): copy
#   scalation/RegularizedRegression_AutoMPG.scala into
#   scalation_2.0/src/main/scala/scalation/modeling/ and run, from scalation_2.0:
#   P2_DIR=<path to Project2> sbt "runMain scalation.modeling.regularizedRegression_AutoMPG"
python compare_results.py            # ScalaTion vs statsmodels
python make_figures.py; python make_tables.py
```

References

- [1] R. Quinlan (1993). *Auto MPG*. UCI Machine Learning Repository, <https://doi.org/10.24432/C5859H>.
- [2] A. Hoerl and R. Kennard (1970). Ridge regression: biased estimation for nonorthogonal problems. *Technometrics* 12(1), 55-67.
- [3] R. Tibshirani (1996). Regression shrinkage and selection via the lasso. *J. Roy. Statist. Soc. B* 58(1), 267-288.
- [4] S. Boyd et al. (2011). Distributed optimization and statistical learning via the alternating direction method of multipliers. *Found. Trends Mach. Learn.* 3(1), 1-122.
- [5] J. Friedman, T. Hastie, R. Tibshirani (2010). Regularization paths for generalized linear models via coordinate descent. *J. Stat. Softw.* 33(1), 1-22.
- [6] S. Seabold and J. Perktold (2010). Statsmodels: econometric and statistical modeling with Python. *Proc. 9th Python in Science Conf.*
- [7] J. Miller et al. ScalaTion 2.0, https://github.com/scalation/scalation_2.0.

A ScalaTion output

ScalaTion’s own fit numbers on the training rows (Table 7), then the console listing for each model. The coefficients are on the scaled predictors. The “Std. Error / t / Pr” columns use the no-penalty OLS formulas, so they are not valid for Ridge and Lasso.

Model	R^2	Adj. R^2	SSE	RMSE	MAE	m
Ridge	0.8101	0.8064	3753.2	3.4573	2.6639	314
Lasso	0.8103	0.8065	3749.7	3.4557	2.6576	314
OLS	0.8105	0.8068	3744.2	3.4531	2.6674	314

Table 7: ScalaTion fit on the training rows (m is the number of rows).

```
| RidgeRegression (lambda = 3.394907558380549): ScalaTion native report |
REPORT
  modelName mn = RidgeRegression_3.394907558380549
  parameter b = VectorD(-0.243404, -0.242123, -0.324316, -4.98764, 0.0360111, 2.68490)
SUMMARY
Parameters/Coefficients:
Var      Estimate  Std. Error  t value  Pr(>|t|)  VIF
x0 -0.243404  0.655142   -0.371528  0.710244  NA
x1 -0.242123  0.868305   -0.278845  0.780364  19.303094
x2 -0.324316  0.605501   -0.535617  0.592223  9.386682
x3 -4.987639  0.626904   -7.955982  0.000000  10.062021
x4 0.036011  0.323284    0.111392  0.911306  2.675782
x5 2.684900  0.223099   12.034577  0.000000  1.274317
Residual standard error: 3.496478 on 307.0 degrees of freedom
Multiple R-squared: 0.810078, Adjusted R-squared: 0.806366
F-statistic: 218.24251980417264 on 6.0 and 307.0 DF, p-value: -0.0

| LassoRegression (lambda = 29.042322450462436): ScalaTion native report |
REPORT
  modelName mn = LassoRegression_${lambda}
  parameter b = VectorD(-0.148465, -0.00302684, -0.204847, -5.35127, 0.0597235, 2.67643)
SUMMARY
Parameters/Coefficients:
Var      Estimate  Std. Error  t value  Pr(>|t|)  VIF
x0 -0.148465  0.654834   -0.226721  0.820640  NA
x1 -0.003027  0.867896   -0.003488  0.997217  19.303094
x2 -0.204847  0.605216   -0.338469  0.735010  9.386682
x3 -5.351270  0.626609   -8.540041  0.000000  10.062021
```

```

x4 0.059724 0.323132 0.184827 0.853365 2.675782
x5 2.676426 0.222994 12.002238 0.000000 1.274317
Residual standard error: 3.494834 on 307.0 degrees of freedom
Multiple R-squared: 0.810257, Adjusted R-squared: 0.806549
F-statistic: 218.49616143032688 on 6.0 and 307.0 DF, p-value: -0.0

| Regression (OLS baseline): ScalaTion native report |
REPORT
  modelName mn = Regression_6.0
  parameter b = VectorD(23.6102, -0.193118, 0.0809927, -0.0874492, -5.51686, 0.166326, 2.75129)
SUMMARY
Parameters/Coefficients:
Var      Estimate      Std. Error  t value  Pr(>|t|)  VIF
x0 23.610191 0.197081 119.799674 0.000000 NA
x1 -0.193118 0.654354 -0.295127 0.767897 10.988872
x2 0.080993 0.867261 0.093389 0.925594 19.303094
x3 -0.087449 0.604773 -0.144598 0.885028 9.386682
x4 -5.516858 0.626151 -8.810746 0.000000 10.062021
x5 0.166326 0.322895 0.515108 0.606477 2.675782
x6 2.751292 0.222831 12.347001 0.000000 1.274317
Residual standard error: 3.492277 on 307.0 degrees of freedom
Multiple R-squared: 0.810534, Adjusted R-squared: 0.806831
F-statistic: 218.8911281294038 on 6.0 and 307.0 DF, p-value: -0.0

```

B statsmodels OLS output

```

=====
                        OLS Regression Results
=====
Dep. Variable:          mpg      R-squared:                0.811
Model:                  OLS      Adj. R-squared:            0.807
Method:                 Least Squares      F-statistic:        218.9
Date:                   Tue, 06 Oct 2026      Prob (F-statistic):    9.99e-108
Time:                   17:32:39      Log-Likelihood:       -834.68
No. Observations:       314      AIC:                  1683.
Df Residuals:           307      BIC:                  1710.
Df Model:                6
Covariance Type:        nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
intercept	23.6102	0.197	119.800	0.000	23.222	23.998
cylinders	-0.1931	0.654	-0.295	0.768	-1.481	1.094
displacement	0.0810	0.867	0.093	0.926	-1.626	1.788
horsepower	-0.0874	0.605	-0.145	0.885	-1.277	1.103
weight	-5.5169	0.626	-8.811	0.000	-6.749	-4.285
acceleration	0.1663	0.323	0.515	0.607	-0.469	0.802
model_year	2.7513	0.223	12.347	0.000	2.313	3.190

```

=====
Omnibus:                 35.477      Durbin-Watson:           1.252
Prob(Omnibus):            0.000      Jarque-Bera (JB):        52.934
Skew:                     0.725      Prob(JB):                 3.20e-12
Kurtosis:                 4.394      Cond. No.                 10.8
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.